From Codes to Causes: A Causal Interpretability Audit of Quantized Delta Representations in a Transformer World Model

R. Xia[1*]

[1*]Lancaster University, Lancaster, LA1 4YW, United Kingdom.

Corresponding author(s). E-mail(s): r.xia2@lancaster.ac.uk;

Abstract

World-model agents such as **Δ**-IRIS compress environment dynamics into discrete latent codes, but whether those codes constitute an interpretable—and causally meaningful—vocabulary has not been examined. We present a three-tier interpretability audit of a single fully trained **Δ**-IRIS agent on the Crafter benchmark, progressing from descriptive to informational to interventional evidence. Descriptively, the tokenizer's delta codes form a near-monosemantic event vocabulary: individual codes predict specific achievement unlocks with conditional probabilities up to **0.92** and lifts up to **708×** over base rates. An ablation that removes only the delta and action conditioning—holding codebook size, token count and backbone fixed—collapses this event-selectivity to the level of a plain frame tokenizer and of original IRIS (mean best conditional probability **0.39 → 0.06**), showing that the context-aware delta is what sharpens the codes. Informationally, linear probes reveal a confound we term the *HUD shortcut*: cumulative agent state is decodable from a convolutional embedding of the current frame alone—Crafter renders the inventory on screen—matching or exceeding every transformer block on 16 of 18 such concepts; only short-horizon reward anticipation genuinely requires depth. Interventionally, sparse-autoencoder features extracted from the world model's residual stream can be causally load-bearing: projecting out a single direction in the 512-dimensional residual stream suppresses sampling of the corresponding event's detector codes in imagined rollouts (wood collection: **1.00 → 0.00** first-step detector probability), while supervised probe directions with near-perfect decoding accuracy leave rollouts unchanged—a dissociation between decoding and mechanism. Activation patching further shows that next-code prediction is quasi-Markovian, carried almost entirely by the current step's action and latent tokens. The audit provides a transferable methodology for interpreting discrete world-model representations and cautions against equating decodability with mechanism.

Keywords: world models, interpretability, reinforcement learning, discrete representations, sparse autoencoders, causal analysis

1 Introduction

Model-based reinforcement learning agents increasingly learn inside their own imagination: a learned *world model* predicts future observations, rewards, and terminations, and the control policy is optimized largely or entirely on imagined trajectories [1–4]. A prominent family of such agents tokenizes observations into discrete latent codes and models dynamics autoregressively with a transformer [5]. The recent Δ-IRIS agent [6] refines this recipe with *context-aware tokenization*: instead of encoding each frame in isolation, its tokenizer encodes the *transition* (x_t, a_t, x_{t+1}) into a handful of discrete codes, and its decoder reconstructs x_{t+1} given direct access to (x_t, a_t). The codes are thereby pressured to carry only what is *new*—a compressed description of the frame-to-frame delta.

This design suggests an attractive interpretability hypothesis: if the codes describe changes, and meaningful events in an environment are changes, then the learned codebook should constitute an *event vocabulary*—discrete, enumerable, and potentially human-legible. Whether this hypothesis holds, and more importantly whether such codes (or any representation derived from them) are *causally* implicated in the world model's predictions rather than merely correlated with events, has to our knowledge not been tested.

We test it. Using a single fully trained Δ-IRIS agent (one training seed, one environment) on Crafter [7]—a procedurally generated survival environment with 22 ground-truth achievement signals—we conduct an interpretability audit organised in three tiers of evidentiary strength:

1. **Descriptive.** We characterise codebook utilisation over the agent's entire 10^7-step replay buffer and measure the statistical association between every (slot, code) pair and every achievement, via lift and exact binomial tests (Section 5).
2. **Informational.** We train linear probes for 47 concepts on seven representations spanning the tokenizer codebook, the world model's convolutional frame embedding, and every transformer block, with episode-disjoint evaluation; we extract concept activation vectors (CAVs) [8]; and we train a TopK sparse autoencoder [9, 10] on the world model's residual stream (Sections 6–7).
3. **Interventional.** We measure the causal contribution of individual representation directions by projection ablation—both on single-step prediction and *inside multi-step imagination*—with matched-moment and random-direction controls, and we localise causal signal across (layer, position, token-type) cells by activation patching [11, 12] (Section 8).

Our principal findings:

- **Delta codes form a near-monosemantic event vocabulary, and the delta trick causes it.** Individual codes act as detectors for specific achievements with

conditional probability up to 0.92 and lift up to 708×; the strongest associations are corroborated by probes, dictionary features, and visual inspection (Sections 5, 7). A controlled ablation that removes only the delta+action conditioning—and the original IRIS frame tokenizer—are both ≈ 7× less event-selective, isolating the effect to context-aware delta tokenization rather than discrete quantization in general (Section 5.3).

- **The HUD shortcut.** Probes suggest the transformer maintains episodic state—until a control representation (the CNN embedding of the current frame, with no transformer) matches or exceeds every transformer block on 16 of 18 cumulative-state concepts. Crafter renders the agent's inventory on screen, so "memory" probes are answerable by reading the screen. Three complementary experiments (frame-embedding probes, ablation-recovery analysis, and causal tracing) converge on this conclusion. The only probed capability that genuinely requires transformer depth is short-horizon reward anticipation (Section 6).
- **A read–write asymmetry.** Supervised probe (CAV) directions decode concepts with near-perfect held-out AUROC yet are causally inert under single-direction ablation; unsupervised sparse-autoencoder directions are weaker decoders but causally load-bearing—ablating a single direction in the 512-dimensional residual stream suppresses the corresponding event's detector codes in imagined futures with controls intact (Section 8). To our knowledge this decodability–causality dissociation [13, 14] has not previously been exhibited inside a generative imagination loop; it contrasts instructively with game-playing sequence models in which probe directions *are* causally steerable [15, 16].
- **Quasi-Markovian prediction.** Activation patching over all (block, timestep, token-type) cells shows the causal signal for next-code prediction concentrated at the current step's action and latent tokens; patching any cell of the 20-step history restores essentially nothing (Section 8.3).

Beyond the specific findings, the audit itself—descriptive association, probing with shortcut controls, dictionary learning, then intervention inside imagination—constitutes a transferable methodology for interrogating discrete world-model representations.

2 Related work

World models with discrete tokens. Learning behaviour inside a learned simulator dates at least to recurrent world models [1] and video-prediction-based Atari agents [2]; MuZero [3] couples planning with implicit models, and the Dreamer line [4] optimises actor–critics on imagined latent trajectories. IRIS [5] introduced VQ-VAE-style discrete tokenization [17, 18] of frames combined with an autoregressive transformer dynamics model; Δ-IRIS [6] made tokenization context-aware, encoding stochastic deltas between consecutive frames conditioned on action. Interpretability of RL networks more broadly includes concept probing in AlphaZero [19], feature-level analysis of vision-based policies [20, 21], and—closest in spirit to our interventional tier—causal probing of sequence models trained on board games, where linearly decoded world-state directions are also causally steerable [15, 16], and of maze-solving

transformers, where sparse-autoencoder features carry a causal world model and, suggestively for our results, are easier to activate than to suppress [22]. We recover the latter asymmetry from the opposite direction, in a pixel-based RL agent with a generative imagination loop. Our work is, to our knowledge, the first systematic interpretability study of the discrete *transition* vocabularies learned by IRIS-family world models. For broader context on the methods we use, see recent surveys of mechanistic interpretability for AI safety [23].

Probing and its limits. Linear probes [24] are a standard tool for asking what a representation encodes, but high probe accuracy does not imply the probed information is used by the model [14, 25]. Amnesic probing [13] removes probe-identified subspaces and measures behavioural change—iteratively, via nullspace projection [26], precisely because removing a single probe direction rarely suffices against redundant encodings; this observation will matter when we interpret our own single-direction results. Concept activation vectors [8] aggregate probe directions into concept-level explanations. Our HUD-shortcut analysis adds an environment-specific control that we argue should be standard in RL probing studies: a probe on the *input encoder alone*, which bounds how much of an apparent "memory" result is attributable to information rendered in the observation itself. Related cautions about confounded saliency explanations in RL appear in [21, 27].

Dictionary learning and causal analysis. Sparse autoencoders decompose residual streams into overcomplete sets of sparsely active, often monosemantic features [10, 28, 29], with TopK activation providing direct sparsity control [9, 30]; the approach now scales to production language models [31] and to comprehensive open SAE suites across every layer of a model [32]. Causal mediation and activation patching localise behaviour-relevant computation in transformers [11, 12, 33, 34]. We adapt both tool families from language models to a world model, and extend ablation beyond single-step prediction into the agent's autoregressive imagination loop, where the world model's own sampled codes are fed back recursively.

3 Background: the Δ-IRIS agent

Δ-IRIS [6] comprises three modules trained jointly from an experience replay buffer: a tokenizer, a transformer world model, and a CNN–LSTM actor–critic trained on imagined trajectories. We summarise the parts our audit instruments.

Delta tokenizer. The tokenizer encodes a transition (x_t, a_t, x_{t+1}), with frames $x \in \mathbb{R}^{3\times64\times64}$, by stacking both frames and an action embedding channel-wise and passing them through a convolutional encoder. The resulting latent is reshaped into a 2×2 spatial grid of *slots*, and each slot is vector-quantised [17] against a shared codebook of C=1024 entries (d=64, cosine similarity), yielding four discrete codes

$$\mathbf{c}_t = \left(c_t^0, c_t^1, c_t^2, c_t^3\right) \in \{0, \ldots, 1023\}^4. \tag{1}$$

The decoder reconstructs x_{t+1} from $(x_t, a_t, \mathbf{c}_t)$: crucially, it receives x_t and a_t *directly*, so the $4\log_2 1024 = 40$ bits of code need only carry what cannot be inferred from the previous frame and action—the transition's residual novelty. We write (s_k, c_j) for code j at slot k.

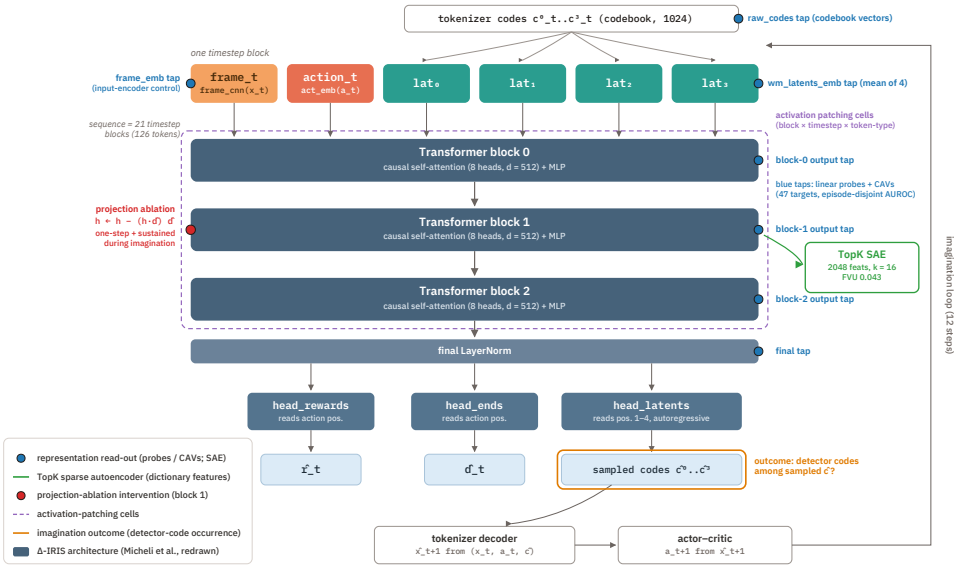

Fig. 1 Overview of the audit. Grey-blue components reproduce the Δ-IRIS world model of Micheli et al. [6] (redrawn from the reference implementation): per timestep, a frame embedding, an action embedding and four tokenizer codes enter a 3-block causal transformer, and three heads predict reward, termination and the next four codes autoregressively. Coloured elements are our contribution—the audit instrumentation: blue taps mark the seven representation read-out points used for linear probes and CAVs (Section 6), including the frame_emb input-encoder control behind the HUD-shortcut analysis; green, the TopK sparse autoencoder trained on the block-1 output (Section 7); red, the projection-ablation intervention point (Equation 2, Sections 8.1–8.2); purple dashes, the activation-patching cell grid (Section 8.3); orange, the imagination-loop outcome (detector-code occurrence among sampled codes). A detailed view of the heads' read positions is given in Appendix B.

Transformer world model. Each timestep contributes a block of six tokens to the world-model sequence: a frame embedding (CNN over x_t), an action embedding, and the four code embeddings. A causal transformer (L=3 blocks, 8 heads, d_{model}=512, context 21 timesteps = 126 tokens) processes the sequence (Figure 1, grey-blue). Three small MLP heads read from fixed intra-block positions (detailed in Appendix B): `head_rewards` and `head_ends` read the action position; `head_latents` reads positions 1–4 (0-indexed within the six-token block) and predicts the four codes of the current transition autoregressively (code c^0 from the action position, c^1 after conditioning on c^0, and so on). During imagination, codes are sampled from `head_latents`, decoded to a frame by the tokenizer, and fed back; the actor–critic selects actions from decoded frames.

Crafter. Crafter [7] is a procedurally generated 2D survival environment with 17 actions and 22 boolean *achievements* (e.g. `collect_wood`, `make_stone_pickaxe`, `defeat_zombie`) that fire once per episode at the step where they are first satisfied, each granting +1 reward. Achievements provide ground-truth event labels at

exact timesteps—ideal supervision for an interpretability audit. Notably for what follows, Crafter renders the agent's inventory and status as a heads-up display (HUD) occupying the lower portion of every frame.

4 Experimental setup

Agent training. We trained a single Δ-IRIS agent with the public reference implementation and unmodified Crafter hyperparameters (1,000 epochs; 10^7 environment steps collected into the replay buffer; seed 0) on one NVIDIA H200 GPU, requiring approximately 8.7 days of wall-clock time. Final performance, evaluated over 500 fresh episodes, is a mean episodic return of 16.20 (s.d. 1.62 across episodes; each of the 22 achievements contributes +1, plus small health-based reward corrections), with 17.1 achievements unlocked per episode on average; the single-seed return is within roughly one seed-level standard deviation of the published multi-seed return of 17.8 ± 1.4 [6]. Achievement coverage is dense: 18 of 22 achievements unlock in at least 73% of evaluation episodes; `collect_diamond` and `eat_plant` never unlock and the two iron tools unlock in ≤ 2 of 500 episodes, so these four are excluded from per-achievement statistics throughout.

Instrumented rollouts. The training pipeline discards the environment info dictionary, so we re-rolled the frozen agent with full instrumentation: at every step we record the four codes $\mathbf{c}_t$ emitted by the frozen tokenizer for the observed transition, the action, the reward, and the 22-bit achievement state, from which we derive *just-unlocked* indicators u_t^i (bit i flipped $0 \to 1$ at step t). Two corpora are used: 500 episodes (294,916 steps) for association statistics, and 150 episodes (81,919 steps) with full frame storage for activation extraction.

Representations probed. For each timestep we extract seven representations: raw_codes (mean of the four codebook vectors, d=64; no transformer); frame_emb (the world model's CNN embedding of the current frame, d=512; no transformer); wm_latents_emb (the world model's own embedding of the four codes, mean-pooled; no attention); and the four transformer stages (output of blocks 0–2 and the final post-norm state), summarised at each timestep by averaging the four latent-token positions. All probe evaluations use episode-disjoint 80/20 train/test splits.

5 Descriptive tier: the codebook is an event vocabulary

5.1 Codebook utilisation

Re-encoding the agent's entire replay buffer (29,136 episodes; 9,980,073 transitions) through the frozen tokenizer shows no codebook collapse: every one of the 1024 codes is used in at least one slot (per-slot active counts 975/972/931/911). Usage is nonetheless heavily concentrated (Figure 2): per-slot entropies are 5.97, 5.96, 5.34 and 5.66 bits of a possible 10, i.e. roughly 40–63 *effective* codes per slot; the single most frequent code per slot absorbs 32–47% of assignments. The same code (index 29) dominates every slot, and visual inspection shows it marks transitions with no change in the corresponding spatial quadrant—a shared "nothing happened here" symbol—consistent with the

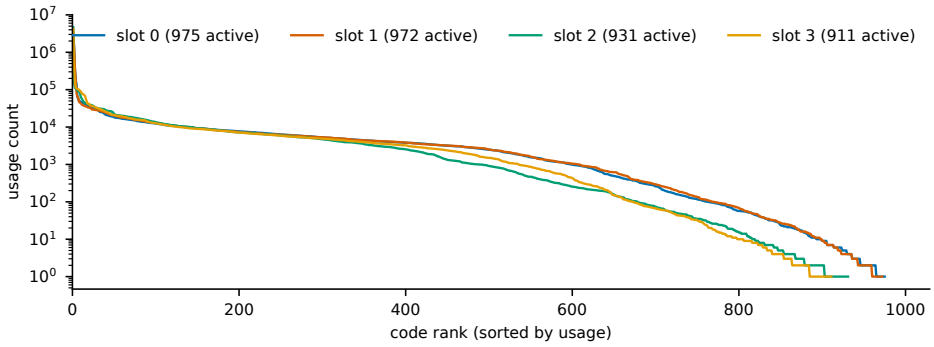

Fig. 2 Codebook utilisation over the full 10^7-transition replay buffer. Per-slot code-usage counts, sorted descending (log scale). All 1024 codes are active in the union of slots, but usage is Zipf-like: the head codes encode "no change in this quadrant" and the event vocabulary occupies the tail.

Table 1 Strongest detector code per achievement (selection). n_{co}: co-occurrences of the code with the just-unlocked event; n_c: total occurrences of the code; lift: $P(u \mid c)/P(u)$. All p-values shown are below 10^{-40} (uncorrected exact binomial tests); the full 431-cell table ships with the code release.

Achievement	Code	n_{co}/n_c	$P(u \mid c)$	Lift
`place_plant`	(s_3, c_{938})	477/519	0.92	543×
`collect_sapling`	(s_2, c_{707})	419/463	0.90	535×
`collect_iron`	(s_3, c_{75})	360/410	0.88	708×
`collect_coal`	(s_3, c_{578})	450/626	0.72	467×
`make_wood_pickaxe`	(s_3, c_{903})	19/35	0.54	325×
`place_table`	(s_3, c_{36})	480/1507	0.32	188×
`eat_cow`	(s_2, c_{648})	251/1127	0.22	134×
`place_furnace`	(s_0, c_{51})	89/412	0.22	132×
`collect_wood`	(s_3, c_{940})	474/2471	0.19	113×
`defeat_zombie`	(s_1, c_{727})	68/365	0.19	113×

sparsity of meaningful change in Crafter: the informative vocabulary lives in the long tail.

5.2 Codes as achievement detectors

For every (slot, code, achievement) cell we count co-occurrences between code assignments and just-unlocked events over the 294,916 instrumented steps, and compute the conditional probability $P(u^i \mid c^k{=}j)$, the lift over the achievement's base rate, and an exact binomial p-value against independence. Of the 431 cells with co-occurrence support ≥ 5, 372 deviate from independence at uncorrected $p < 0.05$, and 190 survive Bonferroni correction over all tested cells; the per-achievement maxima in Table 1 all have $p < 10^{-40}$ and survive any correction.

Table 2 Code→achievement monosemanticity by tokenizer, identical analysis over the same 150-episode labelled set. Per-achievement values are the best detector code's $P(\text{achievement} \mid \text{code})$; columns aggregate over the 18 measurable achievements. The frame-only ablation isolates the delta trick (only delta+action conditioning differs from Δ-IRIS); original IRIS is the literal published baseline.

Tokenizer	mean best $P(a \mid c)$	mean best lift	strong detectors ($P \geq 0.5$)
Δ-IRIS (delta + action)	**0.39**	**234**×	**5 / 18**
frame-only ablation	0.06	34×	0 / 18
original IRIS [5]	0.06	36×	0 / 18

Three observations. First, the strongest codes are bona fide event detectors: when (s_3, c_{75}) fires, the agent is collecting iron 88% of the time, a 708× concentration over chance. Second, slot 3 hosts most of the strongest detectors (11 of 18 per-achievement maxima), indicating slot specialisation beyond pure spatial decomposition. Third, some codes are *polysemantic superclasses*: (s_3, c_{36}) is a shared high-coverage detector for `place_table`, `make_wood_pickaxe` and `make_wood_sword`—it co-occurs with 77–96% of their unlocks (lifts 152–188×), three events whose visual signature (a wooden item appears) is nearly identical—while the rare code (s_3, c_{903}) is the single strongest detector for both wooden tools. Section 7 shows that dictionary learning resolves exactly this collapse. `defeat_skeleton` has by far the weakest detector (best $P(u \mid c) = 0.07$), plausibly because combat produces visually heterogeneous transitions.

5.3 Isolating the delta trick

Is the event-monosemanticity of these codes a product of the *context-aware delta* tokenization specifically, or merely of discrete quantization in general? To answer causally we hold the tokenization scheme fixed and vary only the delta conditioning. We train two baselines on the same 10^7-frame Crafter buffer and run the *identical* code→achievement analysis on all three over a common 150-episode labelled set: (i) a **frame-only ablation** of the Δ-IRIS tokenizer—the same convolutional backbone, codebook (1024), and token count (4), but the encoder sees only x_{t+1} and the decoder reconstructs it from the codes alone (no x_t, no action), so the codes describe the absolute frame rather than the transition; and (ii) the **original IRIS frame tokenizer** [5] (512-entry codebook, 16 tokens/frame, no action), which differs from Δ-IRIS on several axes at once. Both baselines converge to comparable reconstruction error and codebook usage.

The result is unambiguous (Table 2, Figure 3). The delta codes are roughly 7× stronger per-event detectors than either frame baseline in mean best $P(a \mid c)$ (0.39 vs. 0.06) and mean lift (234× vs. 34–36×), and they yield five achievements with a strong ($P \geq 0.5$) single-code detector versus zero for both baselines—an advantage that holds for *every* achievement individually (Figure 3). Crucially, the frame-only ablation and literal IRIS land almost on top of each other (0.056 vs. 0.062), despite their differing codebook sizes, token counts and architectures: the gap to Δ-IRIS is attributable to the delta+action conditioning, not to those incidental differences. The frame tokenizers instead encode absolute scene and inventory content—useful for

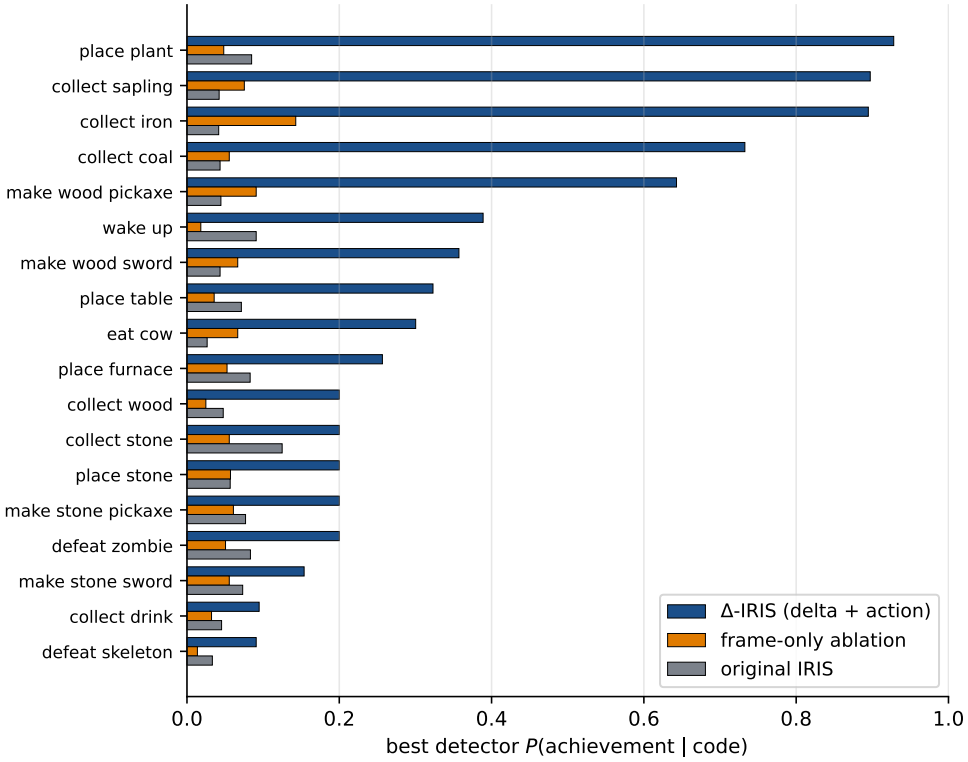

Fig. 3 Per-achievement best detector $P(\text{achievement} \mid \text{code})$ for the Δ-IRIS delta tokenizer, its frame-only ablation, and the original IRIS frame tokenizer, under the identical analysis. Δ-IRIS dominates on every achievement; the frame-only ablation tracks literal IRIS, isolating the gain to the delta+action conditioning rather than codebook/token-count/architecture differences.

cumulative-state decoding (Section 6.1) but not for isolating discrete *events*. Context-aware delta tokenization is what turns the codebook into an event vocabulary.

6 Informational tier: probes, a shortcut, and concept geometry

6.1 Layer-wise probing with an input-encoder control

We train logistic-regression probes for 47 targets—the action taken, instantaneous reward, reward within the next 5 steps, and just-unlocked plus cumulative indicators for each achievement (39 are measurable; the eight indicators of the four never- or rarely-unlocked achievements have no positive examples)—on all seven representations (Figure 4). We report a single episode-disjoint split per representation; AUROCs carry no cross-validated uncertainty estimate.

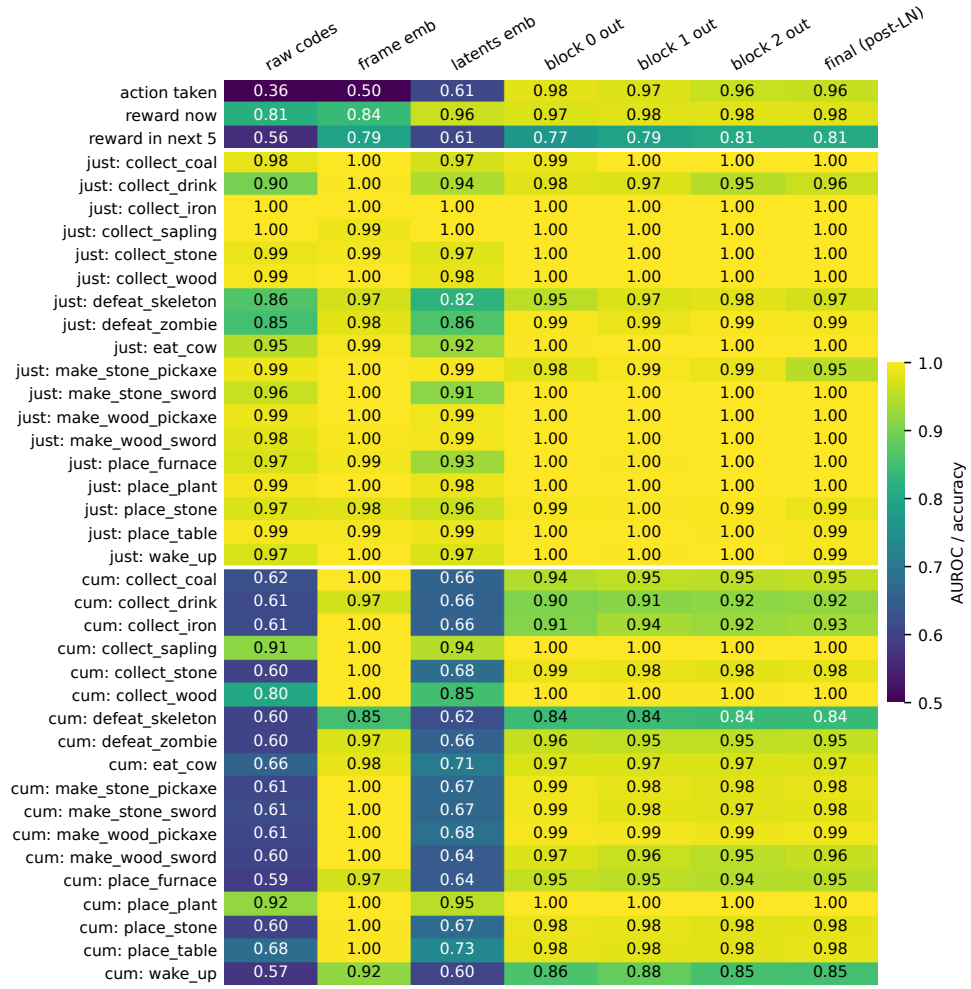

Fig. 4 Held-out probe performance (AUROC; accuracy for the 17-way action probe) for the 39 measurable targets (rows; the remaining 8 of 47 are never positive) across seven representations (columns), episode-disjoint splits, n=81,919 steps from 150 episodes. frame_emb is the input-encoder control: a CNN over the current frame with no transformer. Cumulative-state rows (ach_cum) are answered by frame_emb alone; only reward anticipation (reward_in_next_5) improves materially with transformer depth.

Events are in the codes. Just-unlocked indicators are decodable from raw_codes alone—64 dimensions, no transformer—at AUROC $\geq$ 0.85 for all 18 measurable achievements and $\geq$ 0.95 for 14 of them; the weakest are the two combat events (0.85–0.86), matching their weak detector codes in Section 5.2. The per-sample, held-out view confirms the population-level association analysis.

The HUD shortcut. Cumulative indicators ("has the agent collected wood at any point this episode?") tell a different story. From raw_codes most sit near chance (median AUROC 0.61): delta codes carry little episodic memory, by construction. From any transformer block they look excellent (0.84–1.00), inviting the conclusion that the transformer maintains episodic state. The control column refutes it: frame_emb, which sees only the current frame through a CNN, *matches or exceeds every transformer block on 16 of 18 cumulative concepts* (full-precision comparison)—exceeding AUROC 0.999 on nine of the 18, e.g. `collect_iron` (1.000 vs. 0.935 at block 1). (The two exceptions, `collect_sapling` and `place_plant`, trail by less than 10^{-3}, with frame_emb and every transformer block above 0.999.) The explanation is in the observation, not the network: Crafter draws the inventory in the HUD, so cumulative state is *written on the screen*, and one attention round suffices to relay it from the frame token to the latent positions where our summary is read. Apparent transformer memory is, for these concepts, screen-reading.

What actually needs the transformer. The single probed target where depth helps monotonically by a non-trivial margin and the final block beats frame_emb is reward_in_next_5 (0.56 raw codes; 0.79 frame; 0.77 $\rightarrow$ 0.79 $\rightarrow$ 0.81 $\rightarrow$ 0.81 across blocks): short-horizon *anticipation*, which cannot be read off the current frame. We take this as the cleanest positive evidence of what the world model's depth contributes beyond its input encoders.

We argue the input-encoder control should be standard practice when probing RL agents: most pixel-based environments render persistent state into observations (score counters, HUDs, inventories), making "memory" probes systematically confoundable [25, 27].

6.2 Concept activation vectors and tech-tree geometry

The probe weight vectors are concept activation vectors (CAVs) [8]: directions in representation space pointing toward each concept. Probes are trained on per-feature standardised activations, and the CAV direction used later is the weight vector divided element-wise by the training standard deviations and renormalised, mapping it back to raw activation space. At the block-1 output, 35 of 36 measurable concepts yield CAVs with held-out AUROC ≥ 0.85 (many at 1.0). Their pairwise geometry recapitulates Crafter's dependency structure: the most aligned pairs are exactly the tech-tree neighbours—$\cos(\text{cum}[\texttt{collect\_stone}], \text{cum}[\texttt{place\_stone}]) = 0.95$, $\cos(\text{cum}[\texttt{collect\_sapling}], \text{cum}[\texttt{place\_plant}]) = 0.83$, $\cos(\text{cum}[\texttt{make\_stone\_pickaxe}], \text{cum}[\texttt{make\_stone\_sword}]) = 0.79$—while event and state directions for the same achievement are weakly *anti*-aligned (e.g. $\cos = -0.26$ for `collect_drink`, -0.20 for `collect_wood`), a clean event-versus-state dichotomy. The geometry thus recapitulates the tech tree's co-occurrence structure—though we note cumulative indicators of adjacent achievements co-occur almost perfectly within episodes, which inflates these alignments. Section 8 will show, moreover, that these directions behave as read-out directions only.

Table 3 Cross-corroborated SAE features: the feature's top achievement (by lift within its top activation quartile), the most-aligned CAV, and the (slot, code) with the highest conditional mean activation. The first five rows satisfy the strict triple-corroboration criterion ($|\cos| > 0.4$ to the same-named CAV); f_{372} is included as the third member of the (s_3, c_{36}) split.

Feature	Achievement (lift)	CAV (cos)	Code
f_{1742}	`collect_coal` (589×)	just[`collect_coal`] (+0.58)	(s_3, c_{578})
f_{1820}	`collect_iron` (624×)	just[`collect_iron`] (+0.41)	(s_3, c_{75})
f_{872}	`make_stone_pickaxe` (522×)	just[`make_stone_pickaxe`] (+0.48)	(s_3, c_{616})
f_{677}	`place_table` (527×)	just[`place_table`] (+0.45)	(s_3, c_{36})
f_{1451}	`make_wood_pickaxe` (548×)	just[`make_wood_pickaxe`] (+0.46)	(s_3, c_{36})
f_{372}	`make_wood_sword` (444×)	just[`make_wood_sword`] (+0.23)	(s_3, c_{36})

7 Dictionary learning on the residual stream

We train a TopK sparse autoencoder [9] (M=2048 dictionary features, k=16 active per sample, tied initialisation, unit-norm decoder columns) on the 81,919 block-1 summary activations; TopK fixes sparsity directly, sidestepping the L1 coefficient tuning of earlier SAEs and the JumpReLU variants used in recent large-scale suites [32]. The SAE reconstructs well—fraction of variance unexplained 0.043 on episode-disjoint validation—with zero dead features.

Each feature is then labelled by three independent sources: its best-matching achievement (lift of unlock events among the steps whose activation lies in the top quartile of the feature's nonzero activations), its best-matching CAV (cosine between decoder column and CAV direction), and its best-matching code (mean activation conditional on each (slot, code)). The three labelling sources, computed independently, converge on the same identities for the strongest features (Table 3): five features satisfy a strict triple-corroboration criterion (achievement lift > 10, $|\cos|$ to the *same-named* CAV > 0.4, and a detector code from the MI table as top code).

The last three rows are the noteworthy ones: the SAE assigns *distinct, individually near-monosemantic features* to the single polysemantic code (s_3, c_{36}) identified in Section 5.2—f_{677}, f_{1451} and f_{372} are each the single best feature for `place_table`, `make_wood_pickaxe` and `make_wood_sword` respectively, despite sharing the same top code (eight features in total have (s_3, c_{36}) as their top code). The world model's continuous residual stream retains distinctions that the tokenizer's 40-bit bottleneck collapses, and dictionary learning recovers them [28, 29]. Conversely, the two highest-density features (firing on 38–47% of steps) match no achievement (lift < 3), and none of the 20 features with density above 5% approaches the detector features' lifts (all $\leq 21\times$ vs. $\geq 120\times$): the dense features encode background terrain and idle dynamics, the dictionary analogue of the dominant "no-change" codes.

8 Interventional tier: which directions does the model actually use?

Decodability does not imply mechanism [13, 14]. We therefore measure the *causal* contribution of representation directions by projection ablation,

$$h' \;=\; h - (h^\top \hat{d})\,\hat{d}, \tag{2}$$

applied to the block-1 output h of the world model, where $\hat{d}$ is a unit-normalised candidate direction: the concept's best SAE decoder column, its CAV, or a random direction (specificity control). Effects are always contrasted between *unlock moments* (the 21-step window ends at an achievement unlock) and matched *ordinary moments* from the same episodes.

8.1 Single-step prediction damage

We first ablate at a single timestep and measure $\Delta \log P$, the *drop* in the world model's log-likelihood of the four actually observed codes at the window's final step, computed as baseline − ablated so that a positive value means the ablation lowered the likelihood of the realised codes (n=80 moments per cell; n=26 and 29 for `collect_sapling` and `place_plant`, which have fewer eligible unlock windows; 95% bootstrap CIs throughout; Figure 5). We run the full 2×2 of {matched, random} direction × {unlock, ordinary} moments.

The effect is large but concentrated in a handful of concepts. Ablating the matched SAE direction at the unlock step costs up to +4.85 nats (95% CI $[4.38, 5.38]$; `wake_up`), with `collect_coal` (+2.41), `defeat_skeleton` (+2.20) and `collect_sapling` (+1.73) close behind; these four account for the bulk of the signal (shaded in Figure 5). Eight of 18 concepts show a significant positive effect, but it falls off sharply below the top four. Four concepts show significant *negative* effects (ablation slightly improves prediction, e.g. `collect_iron` −0.34), confirming their best SAE features are off the causal path—a dissociation Section 8.2 revisits. Sorting the rows by effect size (as plotted) should not be read as a smooth gradient across concepts: it is a few large effects atop a near-null tail.

Controls. A random unit direction is inert at both unlock and ordinary moments (all $|\,\text{mean}\,| \leq 0.09$ nats), supporting *direction* specificity. *Moment* specificity is weaker and holds cleanly only for the dominant concepts: for the top four the matched direction barely moves predictions at ordinary moments, but for several mid-table concepts the matched-ordinary effect is comparable to or larger than the matched-unlock effect (e.g. `place_furnace` +0.78 ordinary vs. +0.13 unlock), so the same direction also perturbs the code distribution away from unlock moments. We therefore claim direction specificity broadly and moment specificity only for the dominant concepts.

Two caveats on interpretation. First, $\Delta \log P$ scales with how confidently the model predicts a code: events the model anticipates strongly (e.g. `wake_up`, locked to the day/night cycle) have more likelihood to lose, so effect magnitude partly reflects baseline predictability rather than concept "importance". It is *not* explained by achievement frequency—effect size is uncorrelated with per-step base rate (Pearson

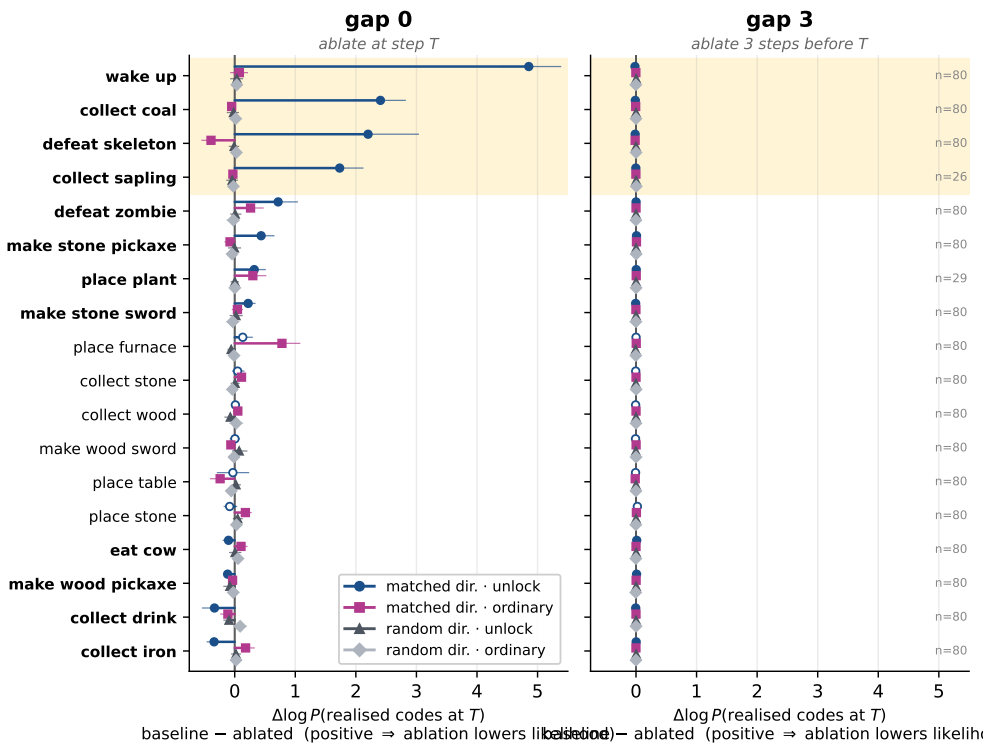

Fig. 5 Single-step causal ablation. $\Delta \log P$ = baseline − ablated log-likelihood of the four *realised* codes at the target step (positive ⇒ the ablation lowered their likelihood) when a direction is projected out of the block-1 residual stream, at the target step (*gap 0*, left) or three steps earlier (*gap 3*, right). Both panels share one horizontal scale, so the gap-3 collapse is read directly (no magnification). Markers are means; whiskers are 95% bootstrap CIs; n moments per condition annotated at right (n=80 except `collect_sapling`/`place_plant`). The full 2×2 of conditions is shown: matched direction at unlock (blue) and ordinary (magenta) moments, random direction at unlock (dark grey) and ordinary (light grey) moments. A filled blue marker / bold row label denotes a matched-unlock CI excluding zero. Rows are sorted by matched-unlock effect; the shaded band marks the four dominant concepts that carry the bulk of the signal. Random directions are inert at both moment types (direction specificity); the matched direction is selective to unlock moments only for the dominant concepts (e.g. matched-ordinary is large for `place_furnace`). Effects vanish entirely at gap 3.

$r = -0.05$; base rates are near-uniform at 0.13–0.18%). Second, this metric measures shifts in predicted *codes*, not whether the achievement actually occurs; Section 8.2 and Appendix C close that gap with rollout-level evidence.

Ablating the same directions three steps *before* the target instead (right panel of Figure 5, shown on the *same* horizontal scale) eliminates the effect entirely (largest mean +0.023 nats, within noise of controls): the model fully recovers within three steps, because the un-ablated frame embeddings of subsequent steps re-inject the relevant information. This is the interventional face of the HUD shortcut.

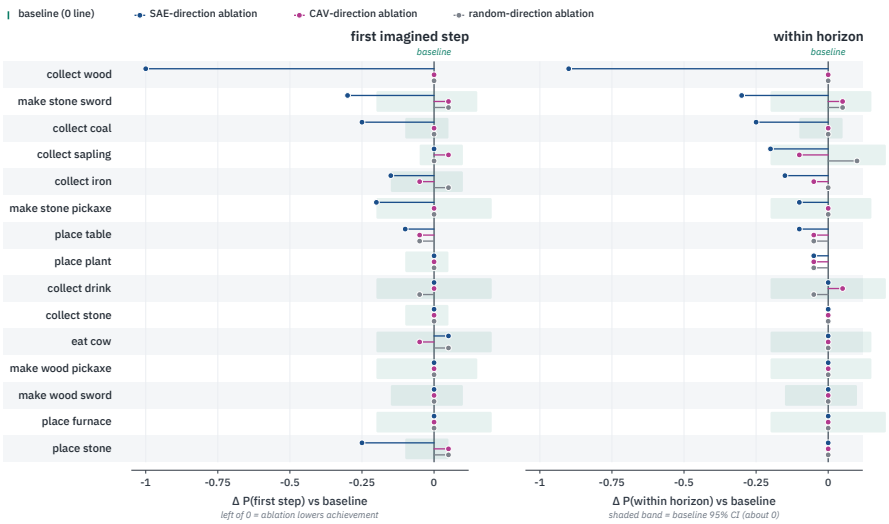

Fig. 6 Concept-direction ablation inside 12-step imagination, for the 15 of 18 concepts whose baseline within-horizon detector probability is at least 0.4 (rows sorted by within-horizon SAE effect). Each point is the *change* in the probability that the concept's detector code is sampled—at the first imagined step (left) and anywhere within the horizon (right)—relative to the no-intervention baseline (the 0 line), under SAE-direction (blue), CAV-direction (magenta) and random-direction (grey) ablation; n=20 rollouts per condition. The shaded band is the baseline 95% CI (mirrored about 0). Ablating the matched SAE direction selectively lowers achievement (blue points fall left of 0, dramatically so for `collect_wood` at -1.0); CAV and random directions stay within baseline noise.

8.2 Ablation inside imagination

Single-step likelihood damage might not matter for behaviour. The stronger test runs inside the agent's imagination loop: burn the world model in on 8 real steps ending just before an unlock, force the real action at the critical step, then let the agent dream 12 steps (codes sampled from `head_latents`, frames decoded by the tokenizer, actions chosen by the actor–critic), with the candidate direction continuously projected out (at every sequence position) during imagination only. For each concept we ablate its highest-lift SAE feature from Section 7. The outcome is whether the achievement's detector codes (top 2 from Section 5.2) appear among the sampled codes (n=20 moments per concept and condition).

Figure 6 shows the headline result. For `collect_wood`, the baseline world model dreams the wood-collection code at the first imagined step in 100% of rollouts; ablating SAE feature f_{187} reduces this to 0% (and to 10% anywhere in the 12-step horizon; Fisher exact $p < 10^{-9}$), while the CAV direction and a random direction leave it at 100%. Removing a single direction in the 512-dimensional residual stream suppresses an event class's codes in the model's imagined futures. Suggestive but individually non-significant decreases in the same pattern (Fisher exact $p = 0.09$–0.32 at n=20) appear for `make_stone_sword` (0.75 → 0.45 within-horizon), `place_stone` (first-step 0.95 → 0.70), `collect_coal` (0.95 → 0.70), `make_stone_pickaxe` (first-step 0.75 → 0.55)

and `collect_iron` (1.00 → 0.85); roughly half the concepts show no effect. `wake_up` (excluded from the figure by its low baseline) shows a nominal increase (0.30 → 0.50; $p = 0.33$), consistent with sampling noise—though it is notable that `wake_up`'s large single-step likelihood effect (Section 8.1) does not translate into imagination suppression, suggesting likelihood damage and rollout-outcome change can dissociate.

The effect is also visible at the level of decoded frames. Appendix C shows paired imagined trajectories for `collect_wood` from a shared burn-in and random seed: under the matched-SAE ablation the dream never produces the wood-collection code and visibly degrades, whereas removing an equally strong but *off-target* SAE feature (the `collect_iron` feature) or a random direction leaves both the event and the rollout intact—a direction-specificity control the single-direction CAV null cannot supply.

Two caveats temper the headline. First, the outcome metric is coupled to the intervention: the ablated feature is selected partly by its association with the same detector codes used as outcome (f_{187}'s top code is (s_3, c_{940}), exactly `collect_wood`'s detector; f_{187} is also *not* among the five strictly triple-corroborated features, its CAV cosine being 0.25). The experiment therefore establishes that the feature direction is causally upstream of those codes' *sampling*; whether the model could still express the event through alternative codes is not ruled out. Second, total imagined reward—an outcome independent of the detector codes—shows no SAE-specific signature for `collect_wood` (baseline 2.77; SAE 2.29; CAV 2.21; random 2.24): all three interventions depress imagined reward similarly, so the SAE-specific effect is confined to the detector-code channel.

Read–write asymmetry. The CAV column is the second headline: across *all* concepts, ablating the supervised probe direction—a direction that decodes the concept with near-perfect AUROC—changes the detector-code outcome by approximately nothing (max $|\Delta| = 0.10$). Within this experiment, directions that suffice to *read* a concept from the residual stream are not the directions the computation *writes through*. The CAV null does admit deflationary explanations: concepts may be encoded redundantly, in which case removing any single supervised direction need not disrupt behaviour—the same observation that motivates iterative nullspace projection [26]; logistic-regression weight directions on standardised but unwhitened activations need not align with class-mean differences; and projection ablation ignores the probe's bias term. Our result therefore shows that single probe directions are not causally sufficient to disrupt the computation here—not that no probe-derived subspace is. With that scoping, the practical corollary stands: explanation methods built on individual probe directions (including TCAV-style attribution) can be causally vacuous even at AUROC 1.0, whereas an unsupervised dictionary direction—weaker as a classifier—can be causally load-bearing. The contrast with board-game sequence models, where probe directions do steer the model's world state [15, 16], sharpens the question of when each regime obtains; the directional asymmetry we see also echoes the report that SAE features in maze-solving transformers are easier to activate than to suppress [22].

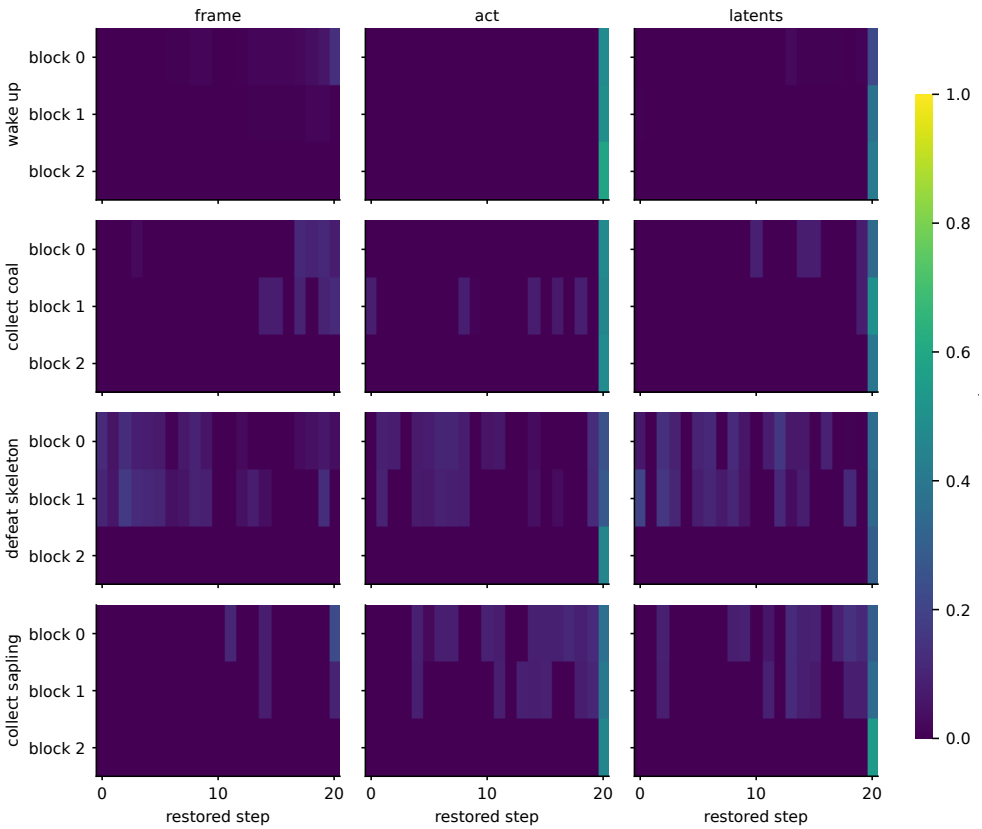

Fig. 7 Causal tracing. Mean restoration of unlock-code prediction when clean activations are patched into a corrupted run, per (block × timestep) cell, for frame, action and latent token positions (columns) and four concepts (rows). Restoration concentrates entirely at the final (unlock) timestep, dominated by the action and latent tokens; patching any historical cell restores essentially nothing.

8.3 Causal tracing

Finally, we localise where the causal signal lives with activation patching [11, 12, 34]. For each unlock window (clean run) we construct a corrupted run from an ordinary window of the same episode, then restore clean activations one cell at a time—each cell being a (transformer block, timestep, token-type) triple with token types {frame, action, latents}—and measure the restoration of the clean codes' log-likelihood, $(\ell_{\text{patched}} - \ell_{\text{corrupt}})/(\ell_{\text{clean}} - \ell_{\text{corrupt}})$, over n=24 window pairs per concept.

The picture (Figure 7) is strikingly concentrated. Patching historical cells restores essentially nothing: mean restoration over the earliest fifteen timesteps is $|\leq 0.04|$ for every (concept, token-type) pair, and the maximum over any single historical cell is 0.20 (an isolated `defeat_skeleton` latents cell at the window's first step). At the final timestep, the action token is the largest carrier on average (restoration 0.46–0.58 at the last block; for two of the four concepts the single largest cell instead belongs

to the latent positions, which reach 0.54), and the frame token contributes least as a single cell (≤ 0.23; its final-block cell is structurally zero, since no later attention layer remains to propagate it to the prediction head's read positions). Next-code prediction is, in causal terms, quasi-Markovian: computed from the current step's action and frame, not from transformer memory of the preceding twenty steps. This is the third complementary line of evidence—after the frame_emb probe control (Section 6.1) and the gap-3 ablation recovery (Section 8.1)—for the same conclusion.

9 Discussion

A triangulated picture of the representation. The three tiers compose into a consistent functional anatomy of Δ-IRIS on Crafter. The tokenizer's delta codes constitute the *event vocabulary*: discrete, sparse, and (in the tail) close to monosemantic. The convolutional frame encoder carries *persistent state*, not because the architecture earmarks it for memory but because the environment renders state on screen. The transformer's distinctive contribution is *anticipation*—integrating recent history into predictions about what comes next—while its apparent episodic memory dissolves under controls. And within its residual stream, the directions that causally carry events are the sparse dictionary directions, not the directions that best decode them.

Methodological lessons. Two of our controls did the heaviest lifting and are cheap to adopt. First, the input-encoder probe (frame_emb) as a shortcut control: without it we would have reported, wrongly but with AUROC 0.98, that the transformer tracks episodic state. Second, the double contrast in ablation (matched vs. ordinary moments × matched vs. random directions): without it, single-direction ablations are uninterpretable. We suggest both as default practice for interpretability studies of RL agents, alongside the broader caution that descriptive and informational evidence—association tables, probe accuracies, concept geometries—should be treated as hypothesis generators for interventions, not as findings about mechanism [14, 27].

Why dictionary directions write and probe directions only read. A linear probe seeks a separator for a label and is free to exploit any correlated signal distributed across many functional directions; nothing constrains it to align with the basis the network computes in. A TopK SAE, in contrast, is trained to *reconstruct* the activation vector from few directions, so its decoder columns must span the signal the downstream computation consumes [9, 29]. Our results are consistent with this folklore distinction in the same model, at the same layer, for the same concepts—but single-direction removal is a weak instrument against redundant codes [26], so the asymmetry should be read as "one SAE direction sufficed where one probe direction did not", pending multi-direction interventions.

Implications for world-model design. The quasi-Markovian finding is double-edged. It demonstrates economy: when the environment exposes state in observations, the model learns not to duplicate it in attention. But it also suggests the 21-step context is underused on Crafter, and that environments with genuinely hidden state would exercise—and let one audit—the transformer's memory in a way Crafter cannot.

10 Limitations

Our audit covers one agent, one seed, one environment. Crafter's HUD, while the source of our central methodological finding, also limits how much the transformer is required to remember; the audit should be repeated in partially observed environments without on-screen state. Four of 22 achievements unlock too rarely for statistics, and combat achievements have weak detector codes, making their imagination-ablation outcomes unreliable. Imagination horizons are capped at 12 steps by the world model's 21-timestep context window. The delta-trick ablation (Section 5.3) trains its two baseline tokenizers standalone for a fixed 40k-step budget, whereas the Δ-IRIS tokenizer co-trained with the agent over the full run; we matched reconstruction convergence but not optimisation steps, so the comparison is at convergence rather than at equal compute. Per-concept interventional samples are modest (n=20–80); only the `collect_wood` imagination effect is individually significant, and the graded effects should be read as suggestive. The SAE was trained at a single layer and width; we did not verify feature stability across dictionary sizes or seeds, and the read–write asymmetry rests on single-direction interventions. Finally, our outcome measure inside imagination—detector-code occurrence—both inherits the imperfections of the detectors and shares provenance with the feature-selection statistic, a circularity we flag explicitly in Section 8.2.

11 Conclusion

We presented a causal interpretability audit—to our knowledge the first for the discrete transition vocabulary of an IRIS-family transformer world model. Δ-IRIS's context-aware quantization yields a genuinely interpretable event vocabulary, validated from population statistics through held-out probes to interventions inside the imagination loop. The audit surfaced two findings with reach beyond this agent: an environment-rendered shortcut that masquerades as transformer memory, detectable with a one-line control; and a read–write asymmetry in which perfectly decoding directions are causally inert under single-direction ablation while a sparse dictionary direction, individually ablated, suppresses the corresponding event's codes in the model's imagined futures. Both argue for the same discipline: in world models as in language models, interpretability claims should climb the ladder from description to intervention before they are believed.

Supplementary information. Interactive HTML artefacts for every experiment (codebook gallery, association tables, probe heatmaps, CAV traces, SAE feature explorer, ablation and tracing reports) are generated by the released code.

Acknowledgements. Experiments were carried out on the High End Computing facility at Lancaster University.

Declarations

- Funding: Not applicable.
- Conflict of interest: The author declares no competing interests.

- Ethics approval and consent to participate: Not applicable.
- Consent for publication: Not applicable.
- Data availability: Result summaries (association tables, probe metrics, ablation effects, tracing grids) are included with the code repository. The trained checkpoint and replay buffer (∼170 GB) are available from the author on reasonable request.
- Materials availability: Not applicable.
- Code availability: All analysis code, Slurm pipelines and figure-generation scripts are available at https://github.com/rxailab/delta-iris-interpretability.
- Author contribution: R.X. designed and performed the experiments and wrote the manuscript.

Appendix A Achievement coverage

Of the 22 Crafter achievements, the evaluation coverage over 500 instrumented episodes is: `place_table` 500, `collect_wood` 500, `place_plant` 499, `collect_sapling` 499, `make_wood_pickaxe` 493, `collect_stone` 492, `eat_cow` 489, `place_stone` 489, `defeat_zombie` 487, `make_wood_sword` 485, `collect_drink` 483, `place_furnace` 482, `make_stone_pickaxe` 478, `wake_up` 471, `collect_coal` 454, `make_stone_sword` 441, `defeat_skeleton` 438, `collect_iron` 366, `make_iron_pickaxe` 2, `make_iron_sword` 2, `collect_diamond` 0, `eat_plant` 0.

Appendix B Hyperparameters and compute

Table B1 Analysis hyperparameters. The agent itself uses the public Δ-IRIS Crafter configuration unchanged.

Component	Setting	Value
Probes	model / split	logistic regression, episode-disjoint 80/20
	samples	81,919 steps / 150 episodes
SAE	dictionary / sparsity	M=2048, TopK k=16
	training	80 epochs, Adam, lr 5×10^{-4}, batch 4096
	layer	block-1 output, latent-position mean
One-step ablation	window / moments	21 steps; n=80 per cell; strength 1.0
Imagination	burn-in / horizon / moments	8 real steps / 12 imagined / n=20
Causal tracing	window / pairs	21 steps; n=24 clean–corrupt pairs
Compute	agent training	≈ 8.7 days, 1× H200
	full analysis pipeline	< 3 hours, 1× H200

Appendix C Imagined trajectories under ablation

Figure C2 renders the imagination-ablation experiment at the level of decoded frames for `collect_wood`. From a shared four-step burn-in ending just before a wood-collection unlock and an identical sampling seed, the agent's imagination is rolled forward 16 steps under four matched conditions; any divergence is therefore

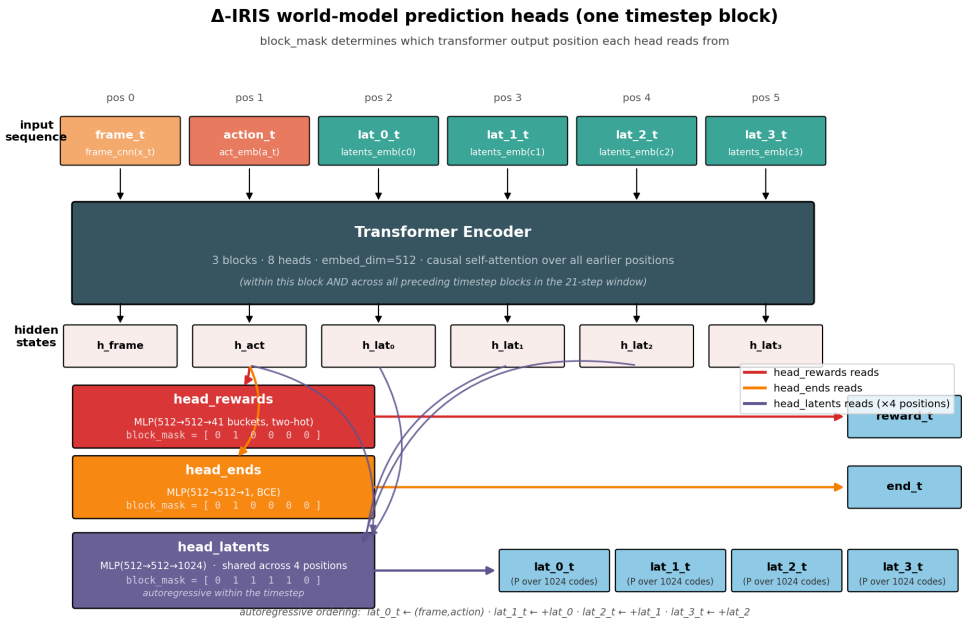

Fig. B1 Detailed view of the Δ-IRIS prediction heads (architecture follows Micheli et al. [6]; our redrawing). `head_rewards` and `head_ends` read the post-transformer activation at the action position (block mask $[0, 1, 0, 0, 0, 0]$); `head_latents` reads positions 1–4 (mask $[0, 1, 1, 1, 1, 0]$) and predicts the four latent codes autoregressively within the timestep. The final latent position feeds subsequent attention but carries no prediction head.

attributable to the intervention rather than to sampling. Ablating the matched SAE feature (f_{187}) suppresses the wood detector code across the entire horizon (0/16, vs. baseline) in 5 of the 5 unlock moments where the event occurs at baseline, and the decoded dream visibly degrades. Removing an off-target SAE feature (the `collect_iron` feature f_{1820}, a live direction of comparable magnitude) or a random unit direction leaves both the wood code and the coherence of the rollout intact—establishing that the effect is specific to the matched concept direction, not a generic consequence of perturbing the residual stream. Two caveats apply: the detector codes are taken from the same association table used to select the SAE feature, so the code metric is not independent of feature selection (the frame panels provide a complementary, non-circular visual readout); and the random control matches direction norm but not removed-energy magnitude, which is why we add the off-target SAE feature as a magnitude-comparable control.

Figure C3 extends the comparison across four concepts and confirms the honest spectrum of outcomes: clean erasure for `collect_wood`, partial suppression for `collect_coal`, and no effect for `collect_iron` or `place_stone`—whose best SAE features are not on the causal path, consistent with the single-step and aggregate results in the main text.

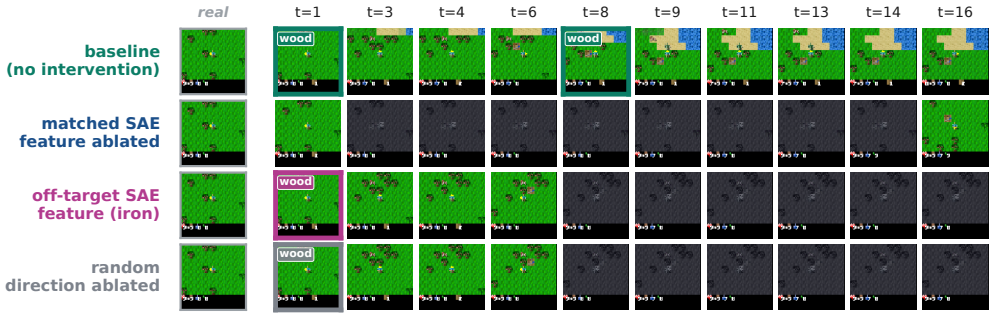

Fig. C2 Paired imagined trajectories for `collect_wood` (decoded frames; *real* = last burn-in frame, then 16 imagined steps, subsampled). All rows share the burn-in and sampling seed. Frames where the wood-collection detector code is sampled are outlined and tagged. Top to bottom: no intervention; matched SAE feature ablated (wood never appears, dream degrades); off-target SAE feature ablated (the `collect_iron` feature—a magnitude-comparable live control—leaves wood and coherence intact); random unit direction ablated (direction-norm control). Only the matched direction erases the event.

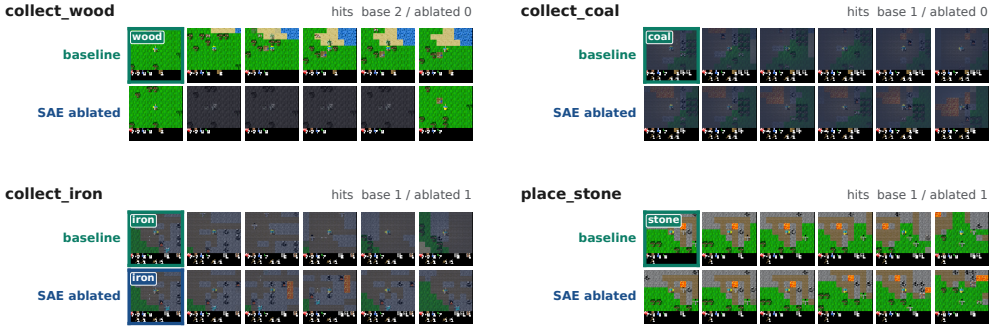

Fig. C3 Imagined trajectories (baseline vs. matched-SAE ablation) for four concepts, with per-row detector-hit counts. The effect ranges from clean erasure (`collect_wood`) through partial suppression (`collect_coal`) to none (`collect_iron`, `place_stone`), matching the aggregate imagination result; the latter two are honest negatives whose best SAE features are not causally load-bearing.

References

[1] Ha, D., Schmidhuber, J.: Recurrent world models facilitate policy evolution. In: Advances in Neural Information Processing Systems 31 (NeurIPS) (2018)

[2] Kaiser, Ł., Babaeizadeh, M., Miłos, P., Osiński, B., Campbell, R.H., Czechowski, K., Erhan, D., Finn, C., Kozakowski, P., Levine, S., Mohiuddin, A., Sepassi, R., Tucker, G., Michalewski, H.: Model based reinforcement learning for Atari. In: The Eighth International Conference on Learning Representations (ICLR) (2020)

[3] Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., Guez, A., Lockhart, E., Hassabis, D., Graepel, T., Lillicrap, T., Silver, D.: Mastering Atari, Go, chess and shogi by planning with a learned model. Nature **588**, 604–609 (2020)

[4] Hafner, D., Pasukonis, J., Ba, J., Lillicrap, T.: Mastering diverse control tasks through world models. Nature **640**, 647–653 (2025)

[5] Micheli, V., Alonso, E., Fleuret, F.: Transformers are sample-efficient world models. In: The Eleventh International Conference on Learning Representations (ICLR) (2023)

[6] Micheli, V., Alonso, E., Fleuret, F.: Efficient world models with context-aware tokenization. In: Proceedings of the 41st International Conference on Machine Learning (ICML) (2024)

[7] Hafner, D.: Benchmarking the spectrum of agent capabilities. In: The Tenth International Conference on Learning Representations (ICLR) (2022)

[8] Kim, B., Wattenberg, M., Gilmer, J., Cai, C., Wexler, J., Viégas, F., Sayres, R.: Interpretability beyond feature attribution: Quantitative testing with concept activation vectors (TCAV). In: Proceedings of the 35th International Conference on Machine Learning (ICML), pp. 2668–2677 (2018)

[9] Gao, L., Tour, T.D., Tillman, H., Goh, G., Troll, R., Radford, A., Sutskever, I., Leike, J., Wu, J.: Scaling and evaluating sparse autoencoders. arXiv preprint arXiv:2406.04093 (2024)

[10] Cunningham, H., Ewart, A., Riggs, L., Huben, R., Sharkey, L.: Sparse autoencoders find highly interpretable features in language models. In: The Twelfth International Conference on Learning Representations (ICLR) (2024)

[11] Vig, J., Gehrmann, S., Belinkov, Y., Qian, S., Nevo, D., Singer, Y., Shieber, S.: Investigating gender bias in language models using causal mediation analysis. In: Advances in Neural Information Processing Systems 33 (NeurIPS) (2020)

[12] Meng, K., Bau, D., Andonian, A., Belinkov, Y.: Locating and editing factual

associations in GPT. In: Advances in Neural Information Processing Systems 35 (NeurIPS) (2022)

[13] Elazar, Y., Ravfogel, S., Jacovi, A., Goldberg, Y.: Amnesic probing: Behavioral explanation with amnesic counterfactuals. Transactions of the Association for Computational Linguistics **9**, 160–175 (2021)

[14] Belinkov, Y.: Probing classifiers: Promises, shortcomings, and advances. Computational Linguistics **48**(1), 207–219 (2022)

[15] Li, K., Hopkins, A.K., Bau, D., Viégas, F., Pfister, H., Wattenberg, M.: Emergent world representations: Exploring a sequence model trained on a synthetic task. In: The Eleventh International Conference on Learning Representations (ICLR) (2023)

[16] Nanda, N., Lee, A., Wattenberg, M.: Emergent linear representations in world models of self-supervised sequence models. In: Proceedings of the 6th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP, pp. 16–30 (2023)

[17] Oord, A., Vinyals, O., Kavukcuoglu, K.: Neural discrete representation learning. In: Advances in Neural Information Processing Systems 30 (NeurIPS) (2017)

[18] Esser, P., Rombach, R., Ommer, B.: Taming transformers for high-resolution image synthesis. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 12873–12883 (2021)

[19] McGrath, T., Kapishnikov, A., Tomašev, N., Pearce, A., Wattenberg, M., Hassabis, D., Kim, B., Paquet, U., Kramnik, V.: Acquisition of chess knowledge in AlphaZero. Proceedings of the National Academy of Sciences **119**(47), 2206625119 (2022)

[20] Hilton, J., Cammarata, N., Carter, S., Goh, G., Olah, C.: Understanding RL Vision. Distill. https://distill.pub/2020/understanding-rl-vision (2020)

[21] Greydanus, S., Koul, A., Dodge, J., Fern, A.: Visualizing and understanding Atari agents. In: Proceedings of the 35th International Conference on Machine Learning (ICML), pp. 1792–1801 (2018)

[22] Spies, A.F., Edwards, W., Ivanitskiy, M.I., Skapars, A., Räuker, T., Inoue, K., Russo, A., Shanahan, M.: Transformers use causal world models in maze-solving tasks. In: The Thirteenth International Conference on Learning Representations (ICLR) (2025)

[23] Bereska, L., Gavves, E.: Mechanistic interpretability for AI safety—a review. Transactions on Machine Learning Research (TMLR) (2024)

[24] Alain, G., Bengio, Y.: Understanding intermediate layers using linear classifier probes. arXiv preprint arXiv:1610.01644 (2016)

[25] Hewitt, J., Liang, P.: Designing and interpreting probes with control tasks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 2733–2743 (2019)

[26] Ravfogel, S., Elazar, Y., Gonen, H., Twiton, M., Goldberg, Y.: Null it out: Guarding protected attributes by iterative nullspace projection. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), pp. 7237–7256 (2020)

[27] Atrey, A., Clary, K., Jensen, D.: Exploratory not explanatory: Counterfactual analysis of saliency maps for deep reinforcement learning. In: The Eighth International Conference on Learning Representations (ICLR) (2020)

[28] Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., Turner, N.L., Anil, C., Denison, C., Askell, A., Lasenby, R., Wu, Y., Kravec, S., Schiefer, N., Maxwell, T., Joseph, N., Hatfield-Dodds, Z., Tamkin, A., Nguyen, K., McLean, B., Burke, J.E., Hume, T., Carter, S., Henighan, T., Olah, C.: Towards Monosemanticity: Decomposing Language Models With Dictionary Learning. Transformer Circuits Thread. https://transformer-circuits.pub/2023/monosemantic-features (2023)

[29] Elhage, N., Hume, T., Olsson, C., Schiefer, N., Henighan, T., Kravec, S., Hatfield-Dodds, Z., Lasenby, R., Drain, D., Chen, C., Grosse, R., McCandlish, S., Kaplan, J., Amodei, D., Wattenberg, M., Olah, C.: Toy Models of Superposition. Transformer Circuits Thread. https://transformer-circuits.pub/2022/toy_model (2022)

[30] Makhzani, A., Frey, B.: k-sparse autoencoders. In: The Second International Conference on Learning Representations (ICLR) (2014)

[31] Templeton, A., Conerly, T., Marcus, J., Lindsey, J., Bricken, T., Chen, B., Pearce, A., Citro, C., Ameisen, E., Jones, A., Cunningham, H., Turner, N.L., McDougall, C., MacDiarmid, M., Freeman, C.D., Sumers, T.R., Rees, E., Batson, J., Jermyn, A., Carter, S., Olah, C., Henighan, T.: Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet. Transformer Circuits Thread. https://transformer-circuits.pub/2024/scaling-monosemanticity (2024)

[32] Lieberum, T., Rajamanoharan, S., Conmy, A., Smith, L., Sonnerat, N., Varma, V., Kramár, J., Dragan, A., Shah, R., Nanda, N.: Gemma scope: Open sparse autoencoders everywhere all at once on gemma 2. In: Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP, pp. 278–300 (2024)

[33] Olah, C., Cammarata, N., Schubert, L., Goh, G., Petrov, M., Carter, S.: Zoom

In: An Introduction to Circuits. Distill. https://distill.pub/2020/circuits/zoom-in (2020)

[34] Heimersheim, S., Nanda, N.: How to use and interpret activation patching. arXiv preprint arXiv:2404.15255 (2024)